# AI for Medical Imaging

## Master Artificial Intelligence

Hoel Kervadec

Fall 2026

Universiteit van Amsterdam
Instituut voor Informatica

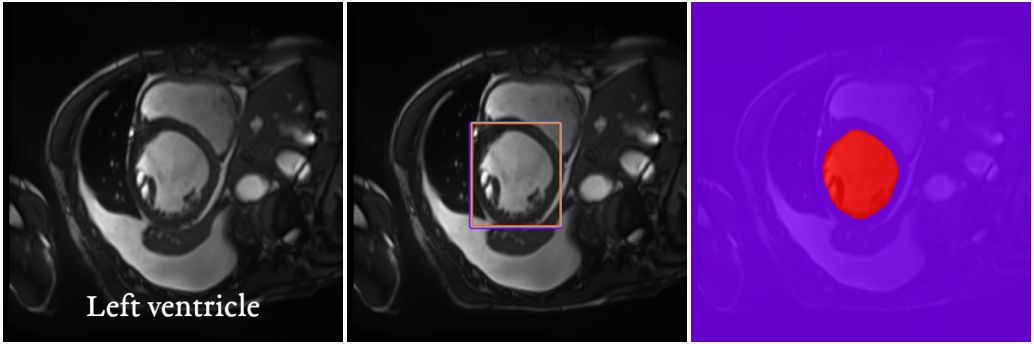
Amsterdam UMC

# 2025/09/03 – Segmentation (Part 1)

# 2025/09/03 – Segmentation (Part 1)

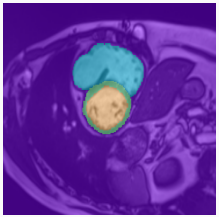
Intro

# Image segmentation

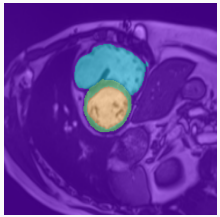



Classification | Object detection | Image segmentation

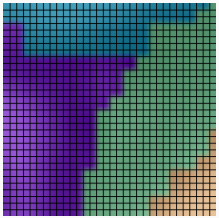
# Image segmentation



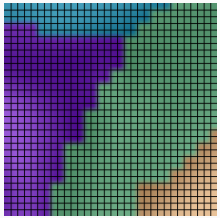
Some label



Some prediction

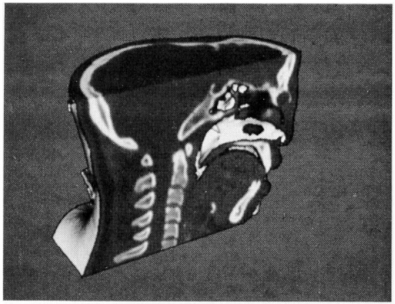


Zoomed label

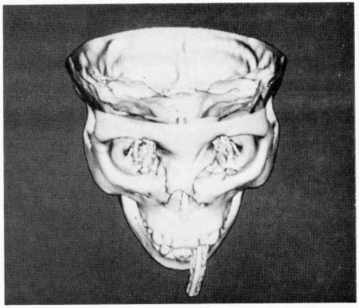


Zoomed prediction

# A bit of history



Input CT

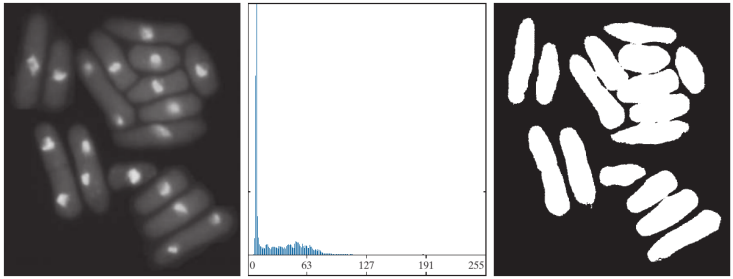


Displayed bones

Lorensen, William E., and Harvey E. Cline. "Marching cubes: A high resolution 3D surface construction algorithm." ACM SIGGRAPH Computer Graphics 21.4 (1987): 163-169.

# Thresholding & gradient based methods

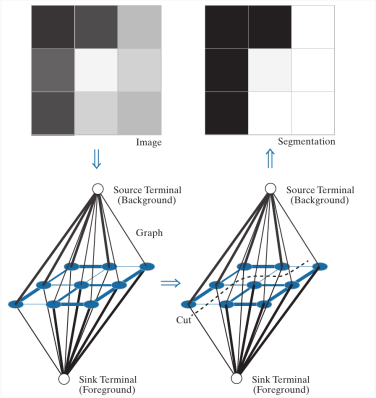
The simplest method to get a segmentation:




Woods, Richard E., and Rafael C. Gonzalez. "Digital Image Processing Fourth Edition." (2019).

Mostly signal processing: not really machine learning

# Graph-based methods




Woods, Richard E., and Rafael C. Gonzalez. "Digital Image Processing Fourth Edition." (2019).

Seminal paper: Boykov, Yuri, Olga Veksler, and Ramin Zabih. "Fast approximate energy minimization via graph cuts." IEEE Transactions on pattern analysis and machine intelligence 23.11 (2001): 1222-1239.

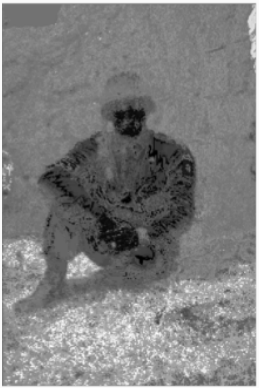
# Limitations

Sensitive to parameters/heuristics

Difficult to parallelize or accelerate with dedicated hardware




Input image



Intensity transform



Raw segmentation



Final segmentation

# 2025/09/03 – Segmentation (Part 1)

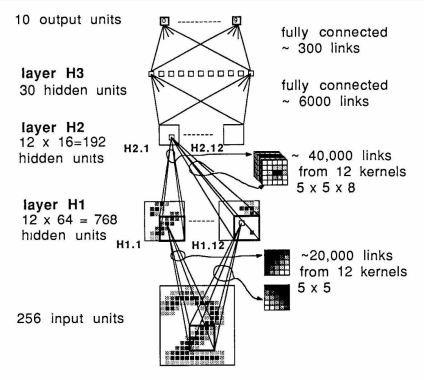
In the deep learning era

# Big data and GPUs to the rescue

"Large" datasets enable to "learn" the features, instead of defining them by hand.

GPU and dedicated hardware enable fast computation, notably with parallel/SIMD operations.

# Convolutional neural networks

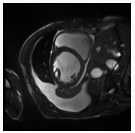


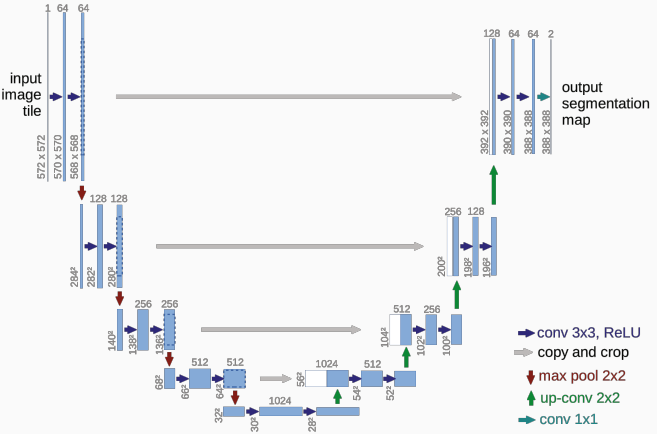

LeCun, Yann, et al. "Backpropagation applied to handwritten zip code recognition." Neural computation 1.4 (1989): 541-551.

Note: input and output are of fixed size.

# Maintaining input size

Use *only* convolutions or size-preserving operations.










Ronneberger, Olaf, Philipp Fischer, and Thomas Brox. "U-net: Convolutional networks for biomedical image segmentation." Medical image computing and computer-assisted

# Other popular architectures

**Fully convolutional networks for semantic segmentation**

*Long, Jonathan, Evan Shelhamer, and Trevor Darrell.* Proceedings of the IEEE conference on computer vision and pattern recognition. 2015.

**Deeplab: Semantic image segmentation with deep convolutional nets, atrous convolution, and fully connected crfs**

*Chen, Liang-Chieh, et al.* IEEE transactions on pattern analysis and machine intelligence 40.4 (2017): 834-848.

**Enet: A deep neural network architecture for real-time semantic segmentation**

*Paszke, Adam, et al.* arXiv preprint arXiv:1606.02147 (2016).

# nnU-Net: the current state-of-the-art

**nnU-Net: a self-configuring method for deep learning-based biomedical image segmentation**

*Isensee, Fabian, et al.* Nature methods 18.2 (2021): 203-211.

- A deep learning-based segmentation method that automatically configures itself, including preprocessing, network architecture, training and post-processing for any new task.
- The key design choices in this process are modeled as a set of fixed parameters, interdependent rules and empirical decisions.

https://github.com/MIC-DKFZ/nnUNet

# High-level overview

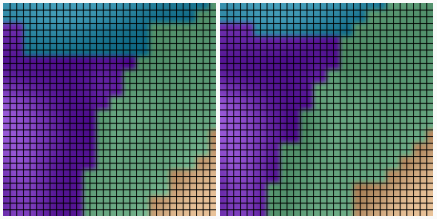
Segmentation overview:

1. pre-processing: ≈ `slice_segthor.py` (Assignment 1)
2. *data loading*: ≈ `dataset.py`
3. **forward and backward propagation**: ≈ `main.py` and `losses.py` (Project)
4. post-processing (optionnal): ≈ `stitching.py` (Assignment 3)

# 2025/09/03 – Segmentation (Part 1)

Formally defining segmentation

# Formal definition (i)



$Y$ overlayed over $x$ $S$ overlayed over $x$

$\Omega \subset \mathbb{R}^D$: image space of dimension $D$

$x \in \mathbb{R}^{M \times \Omega}$: input image with $M$ modalities

$\mathcal{K} = \{0 : \texttt{background}, 1 : \texttt{class-1}, ..., K : \texttt{class-K}\}$: set of K+1 classes to predict

$Y \in \mathcal{K}^{\Omega}$: ground-truth, $S \in \mathcal{K}^{\Omega}$ prediction

# Formal definition (ii)

$\theta$: network parameters to "learn"

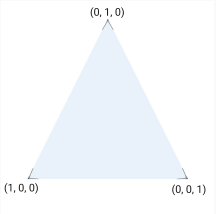
$\mathcal{N}(x;\theta) \in \mathbb{R}^{(K+1)\times\Omega}$: raw output of the neural network

$Y$, $S$ difficult to work with

$$Y \in \mathcal{K}^{\Omega} \underset{\text{one-hot}}{\longmapsto} y \in \left(\widehat{\Delta}^{K}\right)^{\Omega} \subset \{0,1\}^{(K+1)\times\Omega}$$

$$\mathcal{N}(x;\theta) \underset{\text{softmax}}{\longmapsto} s_\theta \in \left(\Delta^{K}\right)^{\Omega} \subset [0,1]^{(K+1)\times\Omega}$$

$$s_\theta \in \left(\Delta^{K}\right)^{\Omega} \underset{\text{argmax}}{\longmapsto} S \in \mathcal{K}^{\Omega}$$

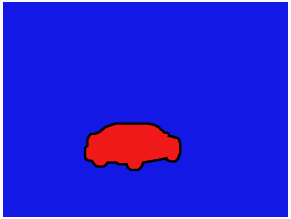



2-simplex $\Delta^2$

# *Training* of deep networks



Input image $x$



Annotated mask $y$

Repeatedly show annotated examples to the network. Optimization problem:

$$\min_{\theta} \quad \mathcal{L}(s_\theta, y),$$

- $\theta$ network parameters;
- $s_\theta$ network output;
- $y$ annotated mask.

# Back-prop from loss

**Back-propagation:** use the chain-rule to compute the gradient :

$$\nabla\theta = \frac{\partial \mathcal{L}(s_\theta, y)}{\partial \theta} = \frac{\partial \mathcal{L}(s_\theta, y)}{\partial s_\theta} \frac{\partial s_\theta}{\partial l_L} \frac{\partial l_L}{\partial l_{L-1}} \cdots \frac{\partial l_1}{\partial l_0}.$$

Parameters update: $\theta_{t+1} = \theta_t - \gamma \nabla \theta_t$,

$\gamma$: learning rate

Variants: SGD, w/ or w/out momentum, Adam, ...:

https://pytorch.org/docs/stable/optim.html#algorithms

# Loss functions for image segmentation

A loss is a formal way to define our objective.

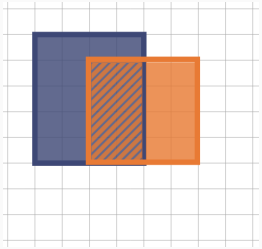
For instance, cross-entropy:

$$\min_{\theta} \mathcal{L}_{\text{CE}}(s_\theta, y) = \sum_{i \in \Omega} \sum_{k \in \mathcal{K}} -y^{(i,k)} \log \left( s_\theta^{(i,k)} \right),$$

| | |
|---|---|
| $\theta$ | network parameters; |
| $y^{(i,k)} \in \{0, 1\}$ | labels; |
| $s_\theta^{(i,k)} \in [0, 1]$ | network output (*softmaxes*); |
| $i \in \Omega$ | image space; |
| $k \in \mathcal{K}$ | set of predicted classes. |

« You should call it entropy, for two reasons. In the first place your uncertainty function has been used in statistical mechanics under that name, so it already has a name. In the second place, and more important, no one really knows what entropy really is, so in a debate you will always have the advantage. » — John von Neumann to Claude Shannon on his new uncertainty function

# Sørensen–Dice coefficient



https://metrics-reloaded.dkfz.de/

$$\mathrm{DSC}(y, s; k) := \frac{2\left|\Omega_y^{(k)} \cap \Omega_s^{(k)}\right|}{\left|\Omega_y^{(k)}\right| + \left|\Omega_s^{(k)}\right|}$$

| | |
|---|---|
| $s$ | discrete segmentation; |
| $\Omega_s^{(k)} \subseteq \Omega$ | subset predicted as $k$; |
| $\Omega_y^{(k)} \subseteq \Omega$ | subset labeled as $k$. |

# DSC as a loss

$$\begin{aligned}\mathrm{DSC}(y, s; k) &:= \frac{2\left|\Omega_y^{(k)} \cap \Omega_s^{(k)}\right|}{\left|\Omega_y^{(k)}\right| + \left|\Omega_s^{(k)}\right|} \\ &= \frac{2\sum_{i\in\Omega} y^{(i,k)} s^{(i,k)}}{\sum_{i\in\Omega}\left[y^{(i,k)} + s^{(i,k)}\right]}\end{aligned}$$

$$\mathcal{L}_{\mathrm{DSC}}(y, s_\theta) := \frac{1}{|\mathcal{K}|}\sum_{k\in\mathcal{K}}\left(1 - \frac{2\sum_{i\in\Omega} y^{(i,k)} s_\theta^{(i,k)}}{\sum_{i\in\Omega}\left[y^{(i,k)} + s_\theta^{(i,k)}\right]}\right)$$

| | |
|---|---|
| $y$ | discrete ground truth |
| $y^{(i,k)} \in \{0, 1\}$ | |
| $s$ | discrete segmentation |
| $s^{(i,k)} \in \{0, 1\}$ | |
| $s_\theta$ | predicted probabilities |
| $s_\theta^{(i,k)} \in [0, 1]$ | |
| $\mathcal{K}$ | set of classes |
| $\Omega$ | image space |

# Derivation of the Dice loss i

$$\frac{\partial \mathcal{L}_{\text{DSC}}}{\partial s_\theta^{(i,k)}} = \frac{\partial \left( -\frac{2 \sum_{i \in \Omega} y^{(i,k)} s_\theta^{(i,k)}}{\sum_{i \in \Omega} \left[ y^{(i,k)} + s_\theta^{(i,k)} \right]} \right)}{\partial s_\theta^{(i,k)}} = \frac{\partial \left( -\frac{2 I^{(k)}}{U^{(k)}} \right)}{\partial s_\theta^{(i,k)}}$$

$$= -2 \left( \frac{\frac{\partial I^{(k)}}{\partial s_\theta^{(i,k)}} U^{(k)} - I^{(k)} \frac{\partial U^{(k)}}{\partial s_\theta^{(i,k)}}}{\left( U^{(k)} \right)^2} \right), \tag{1}$$

with $I^{(k)} = \sum_{i \in \Omega} y^{(i,k)} s_\theta^{(i,k)}$ and $U^{(k)} = \sum_{i \in \Omega} \left[ y^{(i,k)} + s_\theta^{(i,k)} \right]$.

# Derivation of the Dice loss ii

Reminding that $\frac{\partial s_\theta^{(j,k)}}{\partial s_\theta^{(i,k)}} = 1$ if $i = j$ and 0 otherwise, we have:

$\frac{\partial I^{(k)}}{\partial s_\theta^{(i,k)}} = \sum_{j \in \Omega} \frac{\partial y^{(j,k)} s_\theta^{(j,k)}}{\partial s_\theta^{(i,k)}} = y^{(i,k)}$ and $\frac{\partial U^{(k)}}{\partial s_\theta^{(i,k)}} = \sum_{j \in \Omega} \frac{\partial s_\theta^{(j,k)}}{\partial s_\theta^{(i,k)}} = 1$. Hence:

$$\frac{\partial \mathcal{L}_{\text{DSC}}}{\partial s_\theta^{(i,k)}} = \frac{-2\left(y^{(i,k)} U^{(k)} - I^{(k)}\right)}{\left(U^{(k)}\right)^2}$$

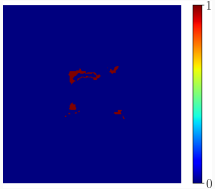
□

# Derivation of the Dice loss iii

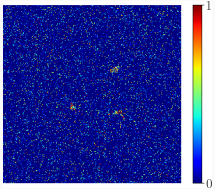
Ultimately:

$$\frac{\partial \mathcal{L}_{\text{DSC}}}{\partial s_\theta^{(i,k)}} = \begin{cases} \frac{-2\left(U^{(k)} - I^{(k)}\right)}{\left(U^{(k)}\right)^2} & \text{if } y^{(i,k)} = 1, \\ \frac{2I^{(k)}}{\left(U^{(k)}\right)^2} & \text{otherwise.} \end{cases} \tag{2}$$

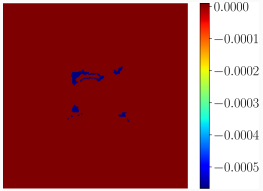
# Gradients



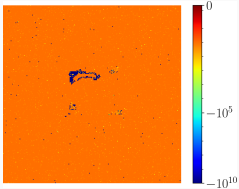

$y^{(1,k)}$




$s_\theta^{(1,k)}$




$$\frac{\partial \mathcal{L}_{\text{DSC}}}{\partial s_\theta^{(0,k)}} + \frac{\partial \mathcal{L}_{\text{DSC}}}{\partial s_\theta^{(1,k)}}$$




$$\frac{\partial \mathcal{L}_{\text{CE}}}{\partial s_\theta^{(0,k)}} + \frac{\partial \mathcal{L}_{\text{CE}}}{\partial s_\theta^{(1,k)}}$$

# Popular mix of losses

Fixed parameter of nnU-Net:

$$\mathcal{L} = \mathcal{L}_{\text{CE}} + \lambda \mathcal{L}_{\text{DSC}}, \quad \lambda > 0$$

$$\nabla\theta = \frac{\partial \mathcal{L}(s_\theta, y)}{\partial \theta} = \frac{\partial \mathcal{L}(s_\theta, y)}{\partial s_\theta} \frac{\partial s_\theta}{\partial l_L} \frac{\partial l_L}{\partial l_{L-1}} \cdots \frac{\partial l_1}{\partial l_0}$$

$$\theta_{t+1} = \theta_t - \gamma \nabla\theta_t$$

Loss function

Network architecture

Optimizer

# Next times

Tuesday 8: Registration

Thursday 10: Segmentation II

Quick exercise: do by hand $\frac{\partial \mathcal{L}_{\text{CE}}(y, \boldsymbol{s})}{\partial \boldsymbol{s}^{(i,k)}}$

# Summary of notation (so far) i

$\Omega \subset \mathbb{R}^D$: image space of dimension $D$

$x \in \mathbb{R}^{M \times \Omega}$: input image with $M$ modalities

$\mathcal{K} = \{0 : \texttt{background}, 1 : \texttt{class-1}, ..., K : \texttt{class-K}\}$: set of K+1 classes to predict

$Y \in \mathcal{K}^\Omega$: ground-truth, $S \in \mathcal{K}^\Omega$ prediction

$\theta$: network parameters to "learn"

$\mathcal{N}(x; \theta) \in \mathbb{R}^{(K+1) \times \Omega}$: raw output of the neural network

$\Delta^K$: $K$-simplex (so $K + 1$ probabilities drawn in a K-dimensional space)

$\widehat{\Delta}^K$: $K$-simplex, restricted to its vertices (one-hot).

# Summary of notation (so far) ii

$Y \in \mathcal{K}^\Omega \underset{\text{one-hot}}{\longmapsto} y \in (\widehat{\Delta}^K)^\Omega \subset \{0,1\}^{(K+1)\times\Omega}$: one-hot encoding of the label $Y$

$\mathcal{N}(x;\theta) \underset{\text{softmax}}{\longmapsto} s_\theta \in (\Delta^K)^\Omega \subset [0,1]^{(K+1)\times\Omega}$: vector of predicted probabilities after softmax layer

$s_\theta \in (\Delta^K)^\Omega \underset{\text{argmax}}{\longmapsto} S \in \mathcal{K}^\Omega \underset{\text{one-hot}}{\longmapsto} s \in (\widehat{\Delta}^K)^\Omega$: representation of the prediction as one-hot vector (useful for indicator functions)

$\mathcal{L}(s_\theta, y)$: some loss

$\mathcal{L}_{\text{CE}}$: cross-entropy loss

$\mathcal{L}_{\text{DSC}}$: Dice loss

$\nabla\theta = \frac{\partial\mathcal{L}(s_\theta,y)}{\partial\theta}$: gradient

# Summary of notation (so far) iii

$\gamma$: learning rate